

Data Analyst Assessment



PEI Technical Round - Bikash Singh

Contact : +91-7439078005 Email : singhbikash136@gmail.com

Task :

<https://docs.google.com/document/d/1zWXz9SViWSJbMN7EmXCyqqFIXzPk-s98lZd45tu3V3o/edit?tab=t.0#heading=h.79boebfspuar>

Data Source : <https://drive.google.com/drive/folders/1cfVJx6licqLNKwWUIJG-O-htpTK3R-tE>

Index

1. Source, Tables, Columns & Description
2. Verify Accuracy, Completeness & Reliability
3. Proposed Table Structure in DE Layer with Technical specifications & instructions
4. Data Model / ER Diagram
5. Reporting Requirements
6. Missing Data : Gaps & Recommendations
7. SQL Queries to Answer Business Requirements with Output results

1. Source, Tables, Columns & Description

Three source files were provided across different formats. The tables below catalogue each source.

1.1 Source Overview

Source File	Format	Rows	Description
Customer.xls	Excel (Legacy .xls)	250	Master customer reference data with demographics
Order.csv	CSV	250	Transactional order records with product and amount
Shipping.json	JSON Array	250	Delivery status per customer shipment

1.2 Column Catalogue

Table: dim_customer (Source: Customer.xls)

Column	Data Type	Sample Value	Description
Customer_ID	Integer	1, 25, 173	Surrogate primary key for customer
First	String (Varchar)	Joseph, Gary	Customer first name
Last	String (Varchar)	Rice, Moore	Customer last name
Age	Integer	43, 71, 18	Customer age in years
Country	String (Varchar)	USA, UK, UAE	Customer's country (3 values: USA, UK, UAE)

Table: fact_order (Source: Order.csv)

Column	Data Type	Sample Value	Description
Order_ID	Integer	1, 2, 3	Surrogate primary key for each order
Item	String (Varchar)	Keyboard, Mouse, Monitor	Product name ordered (8 distinct products)
Amount	Integer	400, 300, 12000	Order value in base currency (assumed USD)
Customer_ID	Integer (FK)	139, 250, 239	Foreign key referencing Customer_ID in dim_customer

Table: fact_shipping (Source: Shipping.json)

Column	Data Type	Sample Value	Description
Shipping_ID	Integer	1, 2, 3	Surrogate primary key for shipment
Status	String (Varchar)	Pending, Delivered	Delivery status (binary: Pending / Delivered)
Customer_ID	Integer (FK)	173, 155, 242	Foreign key referencing Customer_ID in dim_customer

2. Verify Accuracy, Completeness & Reliability of Source Data

2.1 Completeness Check

Table	Column	Null Count	Notes
dim_customer	All columns	0	No missing values in any column
fact_order	All columns	0	No missing values in any column
fact_shipping	All columns	0	No missing values in any column

2.2 Accuracy & Data Sufficiency Checks

Table	Column	Issue	Severity	Count	Example
dim_customer	First (name)	Special characters found in names (!, 0, @, 1 substituted for letters)	HIGH	10	N!cole, R0bert, L@rry, A1cia
dim_customer	Age		ok	0	min=18, max=80
dim_customer	Country		ok	0	USA=101, UK=100, UAE=49
dim_customer	Customer_ID		ok	0	Unique across 250 rows
fact_order	Order_ID		ok	0	Unique across 250 rows
fact_order	Customer_ID		ok	0	All valid FKs
fact_order	Quantity	COLUMN MISSING – cannot compute 'total quantity sold'	CRITICAL	N/A	No Qty column exists
fact_shipping	Shipping_ID		ok	0	Unique across 250 rows
fact_shipping	Status		ok	0	Pending, Delivered
fact_shipping	Order_ID	COLUMN MISSING – no link between shipment and order	CRITICAL	N/A	Cannot join Order — Shipping directly

3. Proposed Table Structure in Data Engineering Layer

Technical Specification to build tables in the Data Engineering Layer

User Story

As a Data Engineer, I want to ingest raw data from three source files (Customer.xls, Order.csv, Shipping.json) into a clean, validated Star Schema DE layer consisting of dim_customer, fact_order, and fact_shipping, so that the reporting team can answer all five business requirements accurately and reliably.

Acceptance Criteria

- All three tables are created in the de_layer schema with correct data types and constraints as specified.
- All foreign key relationships are enforced: fact_order.customer_id → dim_customer.customer_id and fact_shipping.customer_id → dim_customer.customer_id.
- Audit columns dw_created_at and dw_updated_at are populated for all rows.
- is_active defaults to TRUE for all new rows in dim_customer.
- No duplicate natural keys exist in any table after load (customer_id, order_id, shipping_id).

3.1 dim_customer

Column	Data Type	Constraint / Default	Notes
customer_id	INT	NOT NULL, UNIQUE	Natural key from source (Customer_ID)
first	VARCHAR(100)	NOT NULL, CHECK (first_name ~ '^[A-Za-z\s\-\.\,]+\$')	Regex enforces clean alphabetic names
last	VARCHAR(100)	NOT NULL	Customer surname
age	INT	NOT NULL	Customer age
country	VARCHAR(100)	NOT NULL	Customer country
dw_created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit: when record inserted into DW
dw_updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit: when record last updated
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	SCD Type 1 soft-delete flag

3.2 fact_order

Column	Data Type	Constraint / Default	Notes
order_id	INT	NOT NULL, UNIQUE	Natural key from source (Order_ID)
customer_id	BIGINT	NOT NULL, FK → dim_customer(customer_id)	FK to customer dimension
Item	Varchar(max)	NOT NULL,	Item Name
amount	Decimal(12,2)	NOT NULL, CHECK (amount > 0)	Order monetary value
dw_created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit column

3.4 fact_shipping

Column	Data Type	Constraint / Default	Notes
shipping_id	INT	NOT NULL, UNIQUE	Natural key from source (Shipping_ID)
customer_id	INT	NOT NULL, FK → dim_customer(customer_id)	FK to customer dimension
status	VARCHAR(50)	NOT NULL	Delivery status
dw_created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit column

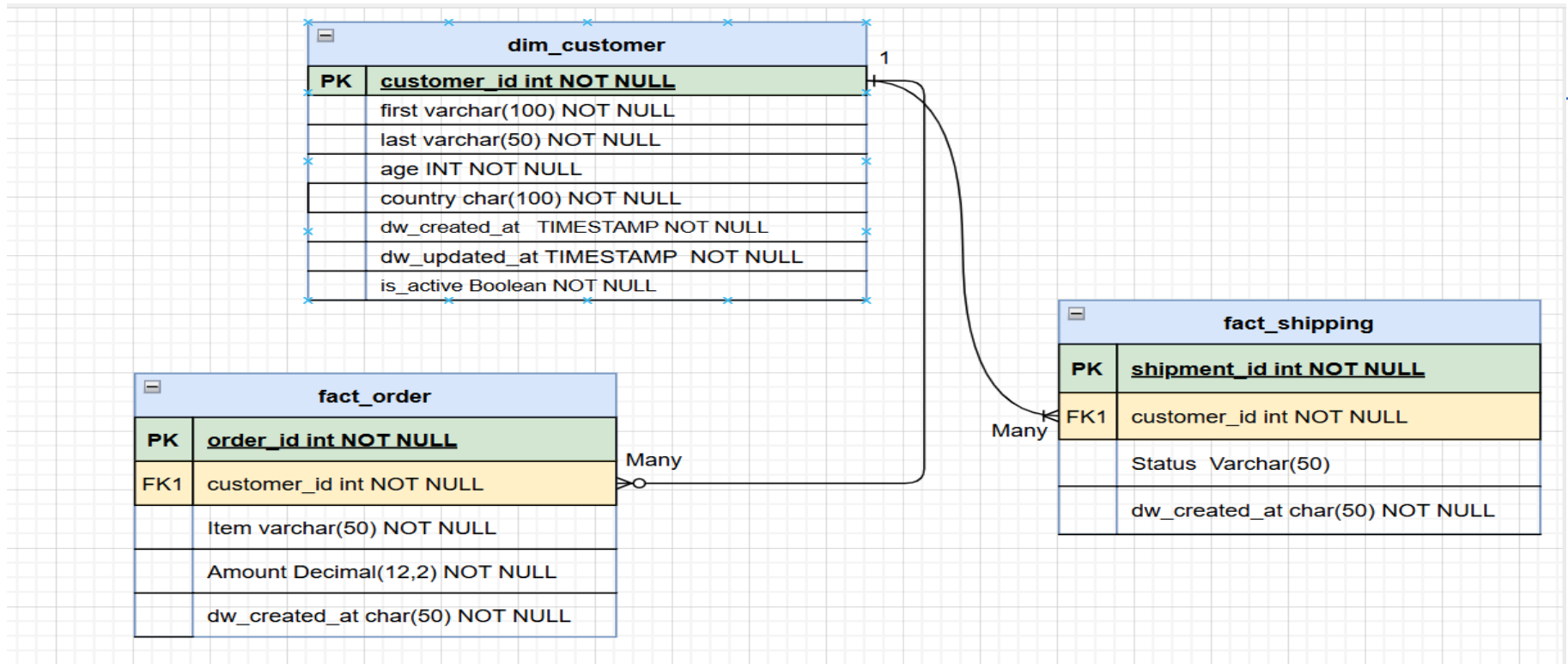
4. Data Model / ER Diagram

The proposed model follows a Star Schema. dim_customer is the central dimension shared by both fact tables.

4.1 Entity Relationship

Entity	Relationship	Entity	Cardinality	Join Key
dim_customer	is purchased by	fact_order	1 : MANY	customer_key
dim_customer	is shipped to	fact_shipping	1 : MANY	customer_key

4.2 Star Schema Layout



5. Reporting Requirements

#	Business Requirement
R1	Total amount spent and the country for the Pending delivery status for each country.
R2	Total number of transactions, total quantity sold, and total amount spent for each customer, along with the product details.
R3	Maximum product purchased per country
R4	Most purchased product for age < 30 and age ≥ 30
R5	Country with minimum transactions and sales amount

6. Missing Data – Gaps & Recommendations

Gap #	Missing Element	Impact	Affected Requirement	Recommendation
G1	Order_ID / Shipping_ID link between fact_order and fact_shipping	Cannot precisely assign a shipment status to a specific order. Joining via Customer_ID creates a many-to-many relationship where a customer with 3 orders and 2 shipments produces 6 rows.	R1, R2	Source system must include Shipping_ID in the order export or Order_ID in the shipping export. This is the most critical gap.
G2	Quantity column in fact_order	R2 explicitly asks for 'total quantity sold'. No quantity data exists; defaulting to 1 per row is an assumption that may be wrong.	R2	Request source team to include Quantity per order line. Assumption c1 unit per order row.
G4	Shipped date and Delivered date in fact_shipping	Cannot compute SLA metrics, average delivery time, or Pending duration.	R1 (SLA context)	Add shipped_date and delivered_date to the shipping source.

Gap #	Missing Element	Impact	Affected Requirement	Recommendation
G5	Currency in fact_order	Amount values are present but currency is unspecified. With customers in USA, UK, UAE, amounts could be in USD, GBP, or AED, making aggregation incorrect.	R1, R2, R5	Add a currency_code column to fact_order. Apply exchange-rate normalisation to a base currency (e.g. USD).
G7	10 customers with special characters in First name	Names like N!cole, R0bert will fail validation and be quarantined, meaning those customers' orders may not be reported.	R2	Source team should fix upstream data entry. Cleansing rule (replace ! → i, 0 → o, @ → a, 1 → i) could be applied as interim fix with business sign-off.

7. SQL Queries to Answer Business Requirements with O/P

Note: Key assumption : Since Quantity column is missing , hence considering each row as 1 qty. Need quantity column in order table

R1 – Total Amount Spent and Country for Pending Deliveries, per Country

Insufficient data to answer this question since there is no order id in shipment table ,Single customer has multiple orders and shipments and we cannot possibly tie particular order to shipment. Many to many relationship will create cartesian product and inflate the values.

R2 – Total Transactions, Quantity Sold, and Amount per Customer with Product Details

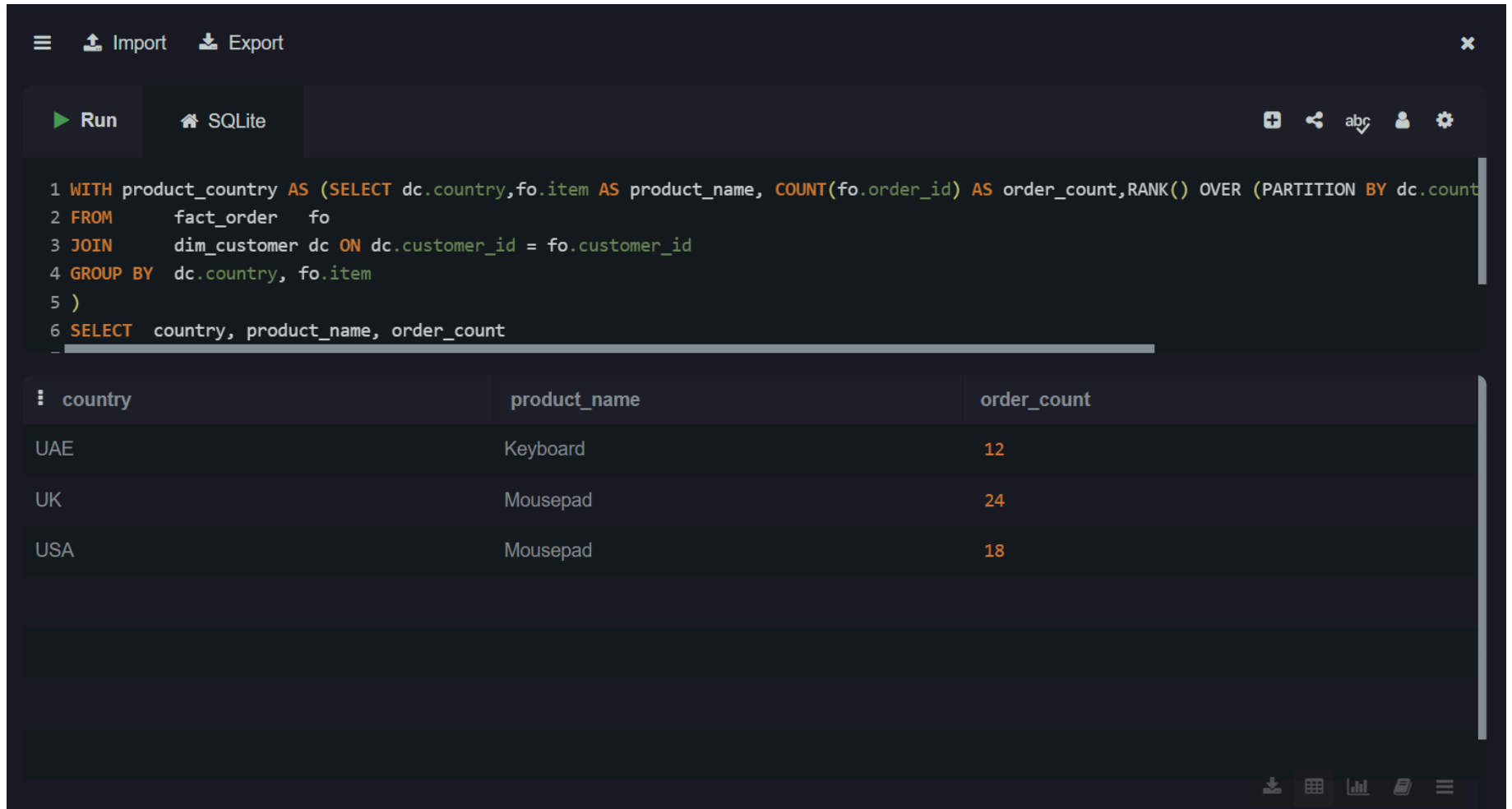
```
SELECT dc.first,dc.last,fo.Item as Product ,COUNT(fo.order_id) as total_transactions,COUNT(fo.order_id) as total_qty,
SUM(fo.amount)as total_amount_spent
FROM    fact_order  fo
JOIN    dim_customer dc ON dc.customer_id = fo.customer_id GROUP BY  dc.first_name, dc.last_name,fo.Item  ORDER BY
total_amount_spent DESC;
```

Import Export Run SQLite abc					
<pre> 1 SELECT dc.first,dc.last,fo.Item AS Product ,COUNT(fo.order_id) AS total_transactions,COUNT(fo.order_id) AS total_qty, 2 SUM(fo.amount)AS total_amount_spent 3 FROM fact_order fo 4 JOIN dim_customer dc ON dc.customer_id = fo.customer_id 5 GROUP BY dc.first, dc.last,fo.Item ORDER BY total_amount_spent DESC; 6 </pre>					
first	last	Product	total_transactions	total_qty	total_amount_spent
Amber	Banks	Monitor	1	1	12000
Chelsea	Moyer	Monitor	1	1	12000
Christy	Rodriguez	Monitor	1	1	12000
Courtney	Lopez	Monitor	1	1	12000
Eric	Harvey	Monitor	1	1	12000
Erin	Taylor	Monitor	1	1	12000
Jade	Wall	Monitor	1	1	12000
Janet	Holmes	Monitor	1	1	12000

Output file link :

https://docs.google.com/spreadsheets/d/1mAeqo7z0ixoQ6RWQfjLv2gjoT2VlhRIO/edit?usp=drive_link&oid=116221171642689647344&rtpof=true&sd=true

R3 – Maximum Product per Country



The screenshot shows a SQL IDE interface with a dark theme. At the top, there are icons for 'Import' and 'Export'. Below that, a 'Run' button and a 'SQLite' database selector are visible. The main area contains a SQL query. Below the query, a table of results is displayed with three columns: 'country', 'product_name', and 'order_count'. The results show three rows: UAE with Keyboard (12), UK with Mousepad (24), and USA with Mousepad (18).

```
1 WITH product_country AS (SELECT dc.country, fo.item AS product_name, COUNT(fo.order_id) AS order_count, RANK() OVER (PARTITION BY dc.country
2 FROM      fact_order  fo
3 JOIN      dim_customer dc ON dc.customer_id = fo.customer_id
4 GROUP BY  dc.country, fo.item
5 )
6 SELECT    country, product_name, order_count
```

country	product_name	order_count
UAE	Keyboard	12
UK	Mousepad	24
USA	Mousepad	18

WITH product_country AS (

SELECT dc.country, fo.item as product_name, COUNT(fo.order_id) AS order_count, RANK() OVER (PARTITION BY dc.country
ORDER BY COUNT(fo.order_id) DESC) AS rnk FROM fact_order fo JOIN dim_customer dc ON dc.customer_key =
fo.customer_key GROUP BY dc.country, fo.item)

SELECT country, product_name, order_count FROM product_country WHERE rnk = 1 ORDER BY country;

R4 – Most Purchased Product by Age Category (< 30 and ≥ 30)

The screenshot shows the SQLiteOnline.com web interface. The browser address bar displays 'sqliteonline.com'. The interface includes a sidebar on the left with a database list: Private.DB, Demo.Memory, SQLite (selected), DuckDB, PGLite, Demo.Server, and MariaDB. The main area shows a SQL query in the editor, a 'Run' button, and the results of the query displayed in a table.

```
1 WITH age_product AS (SELECT CASE WHEN dc.age < 30 THEN 'Under 30' ELSE '30 and Above' END AS age_category,  
2 fo.Item AS product_name , COUNT(fo.order_id) AS order_count, RANK() OVER (PARTITION BY CASE WHEN dc.age < 30 THEN 'Under 30' ELSE '30 and Above' END ORDER BY COUNT(fo.order_id) DESC) AS rn FROM fact_order fo  
3 FROM fact_order fo  
4 JOIN dim_customer dc ON dc.customer_id = fo.customer_id  
5 GROUP BY age_category, fo.Item  
6 )
```

age_category	product_name	order_count
30 and Above	Keyboard	35
Under 30	Mousepad	17

```
WITH age_product AS (SELECT CASE WHEN dc.age < 30 THEN 'Under 30' ELSE '30 and Above' END AS age_category,  
fo.Item as product_name , COUNT(fo.order_id) AS order_count, RANK() OVER (PARTITION BY CASE WHEN dc.age < 30 THEN  
'Under 30' ELSE '30 and Above' END ORDER BY COUNT(fo.order_id) DESC) AS rn FROM fact_order fo  
JOIN dim_customer dc ON dc.customer_id = fo.customer_id GROUP BY age_category, dp.fo.Item)  
SELECT age_category, product_name, order_count FROM age_product WHERE rn = 1  
ORDER BY age_category;
```

R5 – Country with Minimum Transactions and Minimum Sales Amount

The screenshot shows the SQLiteOnline.com web interface. The browser address bar displays 'sqliteonline.com'. The interface includes a top navigation bar with 'Pricing', 'Help', and 'Import/Export' buttons. On the left, there's a sidebar with database options: 'Private.DB +', 'Demo.Memory', 'SQLite' (selected), 'DuckDB', 'PGLite', 'Demo.Server', and 'MariaDB'. The main area shows a SQL query being executed. The query is as follows:

```
1 SELECT dc.country,COUNT(fo.order_id) AS total_transactions,SUM(fo.amount) AS total_sales_amount
2 FROM fact_order fo
3 JOIN dim_customer dc ON dc.customer_id = fo.customer_id
4 GROUP BY dc.country
5 ORDER BY total_transactions ASC,
6 total_sales_amount ASC
```

The results are displayed in a table with the following columns: 'country', 'total_transactions', and 'total_sales_amount'. The results show that the country with the minimum transactions and minimum sales amount is UAE, with 40 transactions and a total sales amount of 49950.

country	total_transactions	total_sales_amount
UAE	40	49950

```
SELECT dc.country,COUNT(fo.order_id) AS total_transactions,SUM(fo.amount) AS total_sales_amount
FROM fact_order fo JOIN dim_customer dc ON dc.customer_key = fo.customer_key
GROUP BY dc.country ORDER BY total_transactions ASC,
total_sales_amount ASC LIMIT 1;
```